
Final Group Assignment

Architectural Evolution of nopCommerce

Curso: AS - Arquitetura de Software

Docentes: Cláudio Teixeira

Aluno: 113655: Abel Teixeira
131224: Francisco Costa
131690: Guilherme Pinho
112901: Tiago Albuquerque

Maio 2026

Conteúdos

1. Contexto	4
1.1 Introdução e contextualização	4
2. Cenários	5
2.1 Os 3 cenários	5
2.2 Cenário escolhido	5
3. Análise ao estado atual	7
3.1 Estrutura arquitetural	7
3.2 Funcionalidades relevantes	7
3.3 Pontos fortes da arquitetura atual	8
3.4 Limitações face ao cenário escolhido	8
3.5 Pontos de evolução	9
3.6 Modelação do Domínio	9
3.7 Bounded contexts	10
3.8 Responsabilidades (Ownership)	10
3.8 Conclusão	10
4. Motivações - de negócio e arquiteturas	12
4.1 Drivers na lógica de negócio	12
4.2 Drivers - Arquitetura	12
5. Quality Attribute Scenarios	13
6. Framework - Desenho arquitetural	15
6.1 ADD (Attribute-Driven Design)	15
7. Arquitetura alvo	16
7.1 Componentes e serviços principais	16
7.2 Data ownership	17
7.3 Interações síncronas e assíncronas	17
7.3.1 Interações síncronas	17
7.3.2 Interações assíncronas	18
7.4 Cross-cutting concerns	18
7.5 Conclusão	18
8. ADRs - Architectural Decision Records	19
8.1 Decisões arquiteturas	19
8.1.1 ADR 1 — Adoção de comunicação assíncrona entre o sistema central e os sistemas externos	19
8.1.2 ADR 2 — Introdução de um Order Integration Service dedicado	19
8.1.3 ADR 3 — Separação de ownership de dados entre contextos	20
8.2 Conclusão	20
9. Plano de risco e estratégias de mitigação	21
9.1 Riscos arquiteturas	21
9.2 Estratégias de mitigação	22
9.3 Conclusão	22
10. Evolution Roadmap	23
10.1 Fases de implementação	23
10.2 Ordem de evolução	24
10.3 Coexistências durante a transição	24
10.4 Conclusão	24

11. Feasibility Spike	25
11.1 Objetivo	25
11.2 Descrição	25
11.3 Resultados esperados	25
11.4 Conclusão	25
12. Conclusão	26

1. Contexto

1.1 Introdução e contextualização

Este tipo de sistemas, como o nopCommerce, têm vindo a evoluir significativamente, deixando de funcionar como plataformas isoladas e passando a integrar ecossistemas mais amplos e interligados. Em contextos empresariais modernos, é comum que diferentes funcionalidades — como gestão de stock, processamento de encomendas, faturação e questões logísticas — estejam distribuídas por múltiplos sistemas especializados.

Neste contexto, pegamos na plataforma open-source - nopCommerce - utilizando-a, novamente, como base para o desenvolvimento deste projeto. Esta plataforma representa uma solução de e-commerce, com uma arquitetura estruturada em camadas, que disponibiliza funcionalidades essenciais como a gestão do catálogo, de compras e de encomendas, assim como, a gestão dos respetivos clientes.

O objetivo deste trabalho consiste em analisar a arquitetura atual da plataforma e explorar possíveis caminhos de evolução que permitam responder a novos desafios de integração, escalabilidade e adaptação a diferentes contextos de negócio.

2. Cenários

2.1 Os 3 cenários

No âmbito deste projeto, são apresentados três cenários distintos, representativos de diferentes perspectivas arquitetônicas no contexto da evolução da plataforma nopCommerce. Cada um destes cenários introduz desafios específicos ao nível da organização do sistema, da integração com outros componentes e da gestão de requisitos não funcionais.

O **Cenário A — Federated Commerce After Acquisitions** aborda um contexto de crescimento empresarial através de aquisições, onde múltiplas unidades de negócio coexistem sob uma mesma estrutura organizacional. Cada unidade mantém particularidades ao nível do catálogo, das operações e dos processos internos, sendo necessário encontrar um equilíbrio entre uma base digital partilhada e a preservação da autonomia local.

O **Cenário B — Regulated / Controlled Commerce** retrata o condicionamento do comércio devido aos requisitos de regulamentação, validação e auditoria. Neste contexto, o sistema deve suportar políticas dinâmicas, processos de aprovação e gestão de evidência documental, garantindo simultaneamente comportamento seguro mesmo quando dependências externas, como sistemas de verificação ou aprovação, apresentam falhas ou inconsistências.

Por fim, o **Cenário C — Omnichannel Commerce Core** foca-se na evolução da plataforma para um papel central num ecossistema mais alargado, onde múltiplos sistemas — como ERP, WMS, serviços de envio e pontos de venda — colaboram para suportar o ciclo completo de uma encomenda. Neste caso, o desafio não reside apenas na integração entre sistemas, mas sobretudo na forma como o núcleo de comércio lida com inconsistências temporárias, atrasos e falhas, mantendo uma experiência coerente e operacional.

De forma geral, os três cenários permitem explorar diferentes dimensões da evolução arquitetural do sistema, desde a distribuição organizacional, passando pela conformidade regulatória, até à integração operacional em ambientes complexos e distribuídos.

2.2 Cenário escolhido

Tendo em conta os 3 cenários apresentados no ponto anterior e, avaliando o objetivo para este trabalho final de grupo, optámos por selecionar o **cenário C - Omnichannel Commerce Core**. Deste modo, o objetivo passa então por evoluir a plataforma nopCommerce, de uma plataforma e-commerce para uma espécie de núcleo central de coordenação de operações no âmbito do comércio.

Atualmente, o nopCommerce funciona principalmente como uma loja online autónoma, sendo responsável por gerir catálogo, carrinho, encomendas e o respetivo estado. No entanto, em contextos empresariais reais, estas responsabilidades encontram-se frequentemente distribuídas por múltiplos sistemas especializados, tais como ERP, sistemas de gestão de armazém (WMS), serviços de shipping e plataformas de ponto de venda (POS).

Posto isto, o principal objetivo, depois de optarmos por este terceiro cenário, é tratar de uma evolução do atual nopCommerce para uma espécie de **commerce coordination core**, capaz de integrar e coordenar múltiplos sistemas externos, mantendo-se funcional e consistente mesmo perante atrasos, falhas ou inconsistências nesses sistemas.

3. Análise ao estado atual

A plataforma nopCommerce apresenta uma arquitetura baseada numa abordagem em camadas. Esta estrutura permite uma separação lógica entre diferentes responsabilidades do sistema, nomeadamente ao nível da apresentação, lógica de negócio e acesso a dados.

3.1 Estrutura arquitetural

De forma geral, a arquitetura do nopCommerce pode ser caracterizada pelas seguintes camadas principais:

- **Presentation Layer:**
Responsável pela interação com o utilizador, incluindo a interface web e os controladores que tratam os pedidos HTTP.
- **Application / Services Layer:**
Contém a lógica de negócio da aplicação, incluindo serviços responsáveis pela gestão de produtos, encomendas, clientes e outros processos fundamentais.
- **Domain Layer:**
Representa as entidades e modelos de negócio centrais do sistema.
- **Data Access Layer:**
Responsável pela persistência de dados garantindo a comunicação com a base de dados.

Esta organização permite uma estrutura relativamente clara e extensível, sendo adequada para este tipo de sistemas.

3.2 Funcionalidades relevantes

No contexto deste projeto, destacam-se as seguintes funcionalidades já suportadas pela plataforma:

- Gestão de catálogo de produtos
- Carrinho de compras e processo de checkout
- Criação e gestão de encomendas
- Gestão de clientes
- Consulta do estado das encomendas pelo utilizador

Estas funcionalidades permitem ao nopCommerce operar todo um ciclo de compra desde a seleção de produtos até à finalização da respetiva encomenda.

3.3 Pontos fortes da arquitetura atual

A arquitetura atual apresenta várias características positivas:

- **Centralização do fluxo de negócio:**
Todo o ciclo de encomenda é gerido internamente, garantindo controlo total sobre o estado da aplicação.
- **Coerência e simplicidade:**
A estrutura baseada num sistema central facilita o desenvolvimento e a compreensão do sistema.
- **Modularidade interna:**
A separação por camadas e serviços permite alguma extensibilidade e organização do código.
- **Maturidade da plataforma:**
Sendo uma solução amplamente utilizada, apresenta estabilidade e um conjunto robusto de funcionalidades.

3.4 Limitações face ao cenário escolhido

Apesar das vantagens mencionadas, a arquitetura atual apresenta limitações relevantes quando considerada no contexto de um ecossistema distribuído e integrado:

- **Dependência de estado interno:**
O sistema assume controlo direto sobre entidades como stock e estado das encomendas, não estando preparado para lidar com múltiplas fontes externas.
- **Integração limitada com sistemas externos:**
A arquitetura não está orientada para integração robusta com sistemas como ERP, WMS ou serviços de shipping.
- **Predominância de fluxos síncronos:**
A maior parte das operações ocorre de forma síncrona, dificultando a gestão de atrasos ou falhas externas.
- **Acoplamento ao sistema central:**
A lógica de negócio encontra-se fortemente concentrada, dificultando a extração de componentes independentes.

3.5 Pontos de evolução

Tendo em conta o cenário selecionado, identificam-se vários pontos de evolução relevantes na arquitetura atual:

- **Introdução de integração assíncrona:**
Necessidade de suportar comunicação baseada em eventos entre o nopCommerce e sistemas externos;
- **Separação de responsabilidades:**
Possibilidade de extrair componentes responsáveis pela integração com sistemas externos, reduzindo o acoplamento ao núcleo da aplicação (sistema central);
- **Gestão de consistência eventual:**
Adaptação do sistema para lidar com estados temporariamente inconsistentes entre diferentes sistemas;
- **Introdução de mecanismos de fiabilidade:**
Implementação de estratégias como retries, idempotência e persistência de eventos;
- **Melhoria da observabilidade:**
Necessidade de monitorizar fluxos distribuídos e identificar falhas em integrações externas.

3.6 Modelação do Domínio

Tendo em conta o cenário selecionado e as limitações identificadas na arquitetura atual, torna-se relevante analisar a organização do domínio do sistema, bem como definir fronteiras claras de responsabilidade entre os diferentes componentes.

Subdomínios relevantes - Cenário Omnichannel

Neste nosso contexto, podem ser identificados os seguintes subdomínios principais:

- **Experiência de Comércio:**
Responsável pela interação com o utilizador, incluindo navegação no catálogo, carrinho de compras, checkout e consulta de encomendas;
- **Gestão de Encomendas:**
Responsável pela criação e gestão do estado lógico das encomendas dentro do sistema;
- **Fulfillment / Gestão de Stock (externo):**
Responsável pela gestão de stock real, reserva de produtos e preparação de encomendas, tipicamente suportado por sistemas de armazém (WMS);
- **Entrega / Shipping (externo):**
Responsável pelo processamento de envios e atualização do estado de entrega das encomendas.

3.7 Bounded contexts

Com base nos subdomínios identificados, é possível definir os seguintes bounded

contexts:

Dentro do nopCommerce

- **Commerce Experience Context:**
Responsável pela experiência do utilizador e interação com o sistema;
- **Order Management Context:**
Responsável pela criação e gestão das encomendas, bem como pela emissão de eventos relevantes para integração externa.

Novos (novos contextos):

- **Order Integration Context:**
Responsável pela coordenação entre o nopCommerce e os sistemas externos, incluindo comunicação, retries e gestão de inconsistência;
- **Warehouse Context (WMS):**
Responsável pela gestão de stock e preparação de encomendas;
- **Shipping Context:**
Responsável pelo processamento de envios e atualização de estados de entrega.

3.8 Responsabilidades (Ownership)

A definição clara de responsabilidades entre contextos é fundamental para garantir desacoplamento e evolução independente do sistema:

- O **nopCommerce** mantém a responsabilidade sobre:
 - experiência do utilizador;
 - criação de encomendas;
 - estado visível ao cliente.
- O **Order Integration Context** é responsável por:
 - coordenação de fluxos entre sistemas;
 - comunicação assíncrona;
 - gestão de retries e consistência eventual.
- O **Warehouse (WMS)** é responsável por:
 - gestão de stock real;
 - preparação de encomendas.
- O **Shipping Context** é responsável por:
 - processamento de envios;
 - atualização do estado de entrega.

3.8 Conclusão

Terminado esta análise, o nopCommerce apresenta uma arquitetura sólida e adequada para o seu propósito original como plataforma de e-commerce autónoma. No entanto, para responder às exigências de um **cenário omnichannel**, torna-se necessário evoluir a arquitetura no sentido de suportar integração com múltiplos sistemas externos, promover desacoplamento e introduzir mecanismos de resiliência e consistência eventual.

No que toca à divisão do domínio efetuada, conseguimos estabelecer fronteiras claras entre responsabilidades, reduzindo o acoplamento entre componentes e criando as bases para uma arquitetura mais modular e orientada à integração.

Concluindo, esta análise permitiu-nos identificar os principais pontos de intervenção arquitetural, que irão servir de base para a definição da arquitetura alvo apresentada nos capítulos seguintes.

4. Motivações - de negócio e arquiteturais

É fundamental identificar os principais fatores que motivam a evolução da arquitetura do sistema, tendo em conta o cenário escolhido. Estes fatores podem ser divididos em **drivers/motivações de negócio** e **drivers/motivações arquiteturais**, sendo que os primeiros representam necessidades do ponto de vista de negócio e os segundos traduzem essas necessidades em requisitos técnicos.

4.1 Drivers na lógica de negócio

Os principais drivers na lógica de negócio identificados são os seguintes:

- **Integração com múltiplos sistemas externos:**
A plataforma deve ser capaz de integrar sistemas como ERP, WMS e serviços de shipping, permitindo suportar operações distribuídas;
- **Experiência unificada para o cliente:**
O utilizador deve ter uma visão consistente do estado das suas encomendas, independentemente dos sistemas envolvidos no seu processamento;
- **Continuidade de operação:**
O sistema deve manter-se funcional mesmo quando existem falhas ou atrasos em sistemas externos;
- **Visibilidade do ciclo de encomenda:**
Deve ser possível acompanhar o estado das encomendas ao longo de todo o processo, incluindo etapas externas ao nopCommerce;
- **Capacidade de evolução:**
A arquitetura deve permitir a integração futura de novos sistemas ou canais sem impacto significativo no núcleo do sistema.

4.2 Drivers - Arquitetura

Com base nos drivers aplicados à lógica de negócio identificados, definem-se os seguintes drivers arquiteturais:

- **Resiliência:**
O sistema deve ser capaz de lidar com falhas e indisponibilidade de sistemas externos sem comprometer a sua funcionalidade;
- **Consistência eventual:**
Deve ser aceite que o estado do sistema pode ser temporariamente inconsistente, sendo garantida a sua correção ao longo do tempo;
- **Integração assíncrona:**
A comunicação entre o nopCommerce e sistemas externos deve ser baseada em mecanismos assíncronos, reduzindo o acoplamento;
- **Desacoplamento entre componentes:**
Os diferentes contextos do sistema devem ter responsabilidades bem definidas e baixo acoplamento entre si;
- **Observabilidade:**
O sistema deve permitir monitorizar e analisar o comportamento das integrações e identificar falhas;
- **Modularidade e extensibilidade:**
A arquitetura deve permitir a evolução do sistema de forma incremental, facilitando a introdução de novos componentes.

5. Quality Attribute Scenarios

Com base nos fatores de negócio e de arquitetura identificados, definem-se os seguintes cenários de qualidade, que irão orientar as decisões de desenho da arquitetura.

QA1 — Resiliência a falhas externas

- **Fonte:**
Sistema de gestão de armazém (WMS).
- **Estímulo:**
Indisponibilidade do sistema durante o processamento de uma encomenda.
- **Ambiente:**
Sistema em funcionamento normal, com encomendas a serem processadas.
- **Resposta:**
A encomenda é registada no nopCommerce, sendo colocada num estado intermédio (ex: “pendente de processamento”). O sistema reencaminha automaticamente a operação para um retry assíncrono, sem bloquear a experiência do utilizador.
- **Métrica:**
Nenhuma perda de encomendas; recuperação automática após restabelecimento do sistema externo.

QA2 — Consistência eventual

- **Fonte:**
Sistema de shipping.
- **Estímulo:**
Atraso na atualização do estado de envio de uma encomenda.
- **Ambiente:**
Após criação da encomenda, com integração ativa com sistemas externos.
- **Resposta:**
O nopCommerce apresenta um estado intermédio ao utilizador, sendo posteriormente atualizado quando a informação chega do sistema externo.
- **Métrica:**
Estado final consistente em tempo aceitável (ex: alguns minutos), sem inconsistências permanentes.

QA3 — Desacoplamento e integração

- **Fonte:**
Equipa de desenvolvimento.

- **Estímulo:**
Integração com um novo sistema externo.
- **Ambiente:**
Sistema em produção.
- **Resposta:**
A integração é realizada através de um componente dedicado, sem necessidade de alterar significativamente o núcleo do nopCommerce.
- **Métrica:**
Alterações localizadas e tempo de integração reduzido e sem alterações significativas ao núcleo do sistema

QA4 — Observabilidade e diagnóstico

- **Fonte:**
Operador técnico.
- **Estímulo:**
Falha na comunicação com um sistema externo.
- **Ambiente:**
Sistema em produção.
- **Resposta:**
O sistema disponibiliza tracing distribuído, métricas e logs que permitem acompanhar o fluxo completo da operação entre o nopCommerce e sistemas externos, identificando pontos de falha ou latência.
- **Métrica:**
Capacidade de identificar a causa da falha em tempo reduzido.

QA5 — Continuidade de operação

- **Fonte:**
Utilizador final.
- **Estímulo:**
Atraso ou falha num sistema externo durante o processo de compra.
- **Ambiente:**
Durante a utilização normal da plataforma.
- **Resposta:**
O sistema mantém-se funcional, permitindo ao utilizador concluir a encomenda, mesmo que algumas operações externas sejam processadas posteriormente.
- **Métrica:**
Operação contínua sem interrupção da experiência do utilizador.

Concluindo este tópico, os cenários de qualidade definidos refletem os principais desafios associados à evolução do nopCommerce para um contexto omnichannel conforme escolhemos, nomeadamente ao nível da resiliência, integração e consistência. Estes cenários irão servir de base para a definição da arquitetura alvo, orientando as decisões de desenho apresentadas nos capítulos seguintes.

6. Framework - Desenho arquitetural

6.1 ADD (Attribute-Driven Design)

Para apoiar a evolução arquitetural proposta, selecionámos o método **ADD (Attribute-Driven Design)**.

Esta escolha justifica-se pelo facto de o projeto partir de um conjunto claro de atributos de qualidade e de necessidades arquitetónicas, nomeadamente resiliência, consistência eventual, integração assíncrona e observabilidade. Neste contexto, o ADD revela-se adequado por orientar o desenho da arquitetura a partir desses fatores, permitindo que as decisões estruturais sejam diretamente influenciadas pelos cenários de qualidade previamente definidos.

Além disso, este método ajusta-se bem ao nosso objetivo central, uma vez que não se pretende redesenhar integralmente a plataforma nopCommerce, mas sim definir uma evolução arquitetural focada, justificando o que deve permanecer no núcleo da aplicação, o que deve ser separado e de que forma os diferentes componentes devem interagir.

Assim, o ADD permite estabelecer uma ligação clara entre:

- os objetivos de negócio;
- os atributos de qualidade identificados;
- a decomposição do sistema em contextos e responsabilidades;
- e as decisões arquiteturais que irão sustentar a arquitetura alvo.

Desta forma, a utilização deste método contribui para um processo de desenho mais estruturado, rastreável e coerente com o nosso problema e respetivos objetivos.

Adicionalmente, ainda que outras abordagens como ACDM ou ADM possam ser consideradas em contextos de evolução arquitetural, o ADD revelou-se mais adequado neste caso por permitir uma ligação mais direta entre os cenários de qualidade e as decisões concretas de desenho.

7. Arquitetura alvo

Novamente olhando para o cenário selecionado, os fatores identificados e os cenários de qualidade definidos anteriormente, a arquitetura alvo proposta procura evoluir o nopCommerce de uma plataforma centrada apenas na experiência de compra online para um núcleo de coordenação de operações de comércio num contexto omnichannel.

Como já vimos, o objetivo não passa por substituir integralmente a arquitetura atual, mas sim por introduzir novos componentes e mecanismos que permitam integrar sistemas externos de forma desacoplada, resiliente e observável.

7.1 Componentes e serviços principais

A arquitetura proposta é composta pelos seguintes elementos principais:

- **nopCommerce:**
Mantém-se como sistema central da solução, sendo responsável pela experiência do utilizador, pela navegação no catálogo, pelo carrinho de compras, posterior checkout e criação de encomendas e pela consulta do respetivo estado;
- **Order Integration Service:**
Componente dedicado à integração entre o nopCommerce e os sistemas externos. É responsável por consumir eventos relevantes do domínio das encomendas, coordenar a comunicação com sistemas externos e gerir mecanismos como retry, idempotência e tratamento de falhas;
- **Message Broker:**
Componente responsável pela comunicação assíncrona entre o nopCommerce e o serviço de integração. Permite reduzir o acoplamento entre componentes, suportando a troca de eventos de forma fiável;
- **Warehouse Management System (WMS):**
Sistema externo responsável pela gestão de stock real, reserva de produtos e pela preparação de encomendas;
- **Shipping Service:**
Sistema externo responsável pelo processamento de envios e atualização do estado de entrega.

7.2 Data ownership

A arquitetura proposta define fronteiras claras de responsabilidade sobre os dados de cada contexto:

- O **nopCommerce** mantém a responsabilidade sobre os dados relacionados com:
 - catálogo apresentado ao utilizador;
 - carrinho de compras;
 - encomendas;
 - estado visível ao cliente.
- O **Order Integration Service** mantém dados técnicos e operacionais associados à integração, tais como:
 - eventos processados;
 - estado de retries;
 - informação de correlação;
 - registos necessários à garantia de idempotência.
- O **WMS** é responsável pelos dados relacionados com:
 - stock real;
 - reserva de produtos;
 - preparação de encomendas.
- O **Shipping Service** é responsável pelos dados associados a:
 - criação de envios;
 - tracking;
 - estado de entrega.

Desta forma, evita-se uma partilha direta da base de dados entre os componentes, garantindo um maior desacoplamento e uma evolução independente.

7.3 Interações síncronas e assíncronas

Como expectável, a arquitetura proposta combina interações síncronas e assíncronas, de acordo com a natureza de cada fluxo.

7.3.1 Interações síncronas

As interações síncronas permanecem associadas à experiência direta do utilizador dentro do nopCommerce, incluindo:

- navegação no catálogo;
- operações sobre o carrinho;
- checkout;
- submissão inicial da encomenda;
- consulta do estado da encomenda.

Estas operações exigem resposta imediata ao utilizador e mantêm-se concentradas no sistema central.

7.3.2 Interações assíncronas

As interações com sistemas externos passam a ser feitas de forma predominantemente assíncrona, nomeadamente:

- emissão de eventos de criação de encomenda;
- comunicação entre nopCommerce e Order Integration Service;
- atualização de stock e fulfillment através do WMS;
- atualização do estado de envio através do Shipping Service.

Este modelo permite que o sistema continue operacional mesmo perante atrasos, falhas temporárias ou indisponibilidade de dependências externas, suportando consistência eventual entre os diferentes contextos.

7.4 Cross-cutting concerns

A arquitetura alvo é influenciada por um conjunto de preocupações, transversais, que têm impacto na modelação do desenho da solução:

- **Resiliência:**
A arquitetura deve tolerar falhas e atrasos em sistemas externos, recorrendo a mecanismos como retries e processamento assíncrono;
- **Consistência eventual:**
Aceita-se que o estado visível no nopCommerce possa não refletir imediatamente o estado final dos sistemas externos, sendo garantida a sua reconciliação ao longo do tempo;
- **Desacoplamento:**
A introdução de um serviço de integração e de um broker de mensagens permite reduzir o acoplamento direto entre o nopCommerce e os sistemas externos;
- **Observabilidade:**
A arquitetura deve incorporar observabilidade através de logs, métricas e tracing distribuído, permitindo acompanhar fluxos e diagnosticar falhas;
- **Extensibilidade:**
A arquitetura deve permitir a integração futura de novos sistemas externos sem necessidade de alterações profundas no núcleo da aplicação;
- **Fiabilidade:**
A comunicação entre componentes deve suportar mecanismos de idempotência, retry e registo de eventos, minimizando perda de mensagens ou processamento duplicado.

7.5 Conclusão

Esta arquitetura proposta mantém o nopCommerce como sistema central da experiência de comércio, mas introduz novos componentes e mecanismos que lhe permitem atuar como núcleo de coordenação num ecossistema distribuído. Desta forma, a solução apresentada responde aos principais desafios impostos pelo cenário que selecionamos, um cenário omnichannel, nomeadamente no que toca a questões como a integração com sistemas externos, a resiliência a falhas, a consistência eventual e a capacidade de evolução futura.

8. ADRs - Architectural Decision Records

8.1 Decisões arquiteturais

De forma a sustentar a arquitetura proposta, foram identificadas as seguintes decisões arquiteturais, documentadas através de ADRs. Estas decisões refletem opções estruturais fundamentais para a evolução do nopCommerce no contexto do cenário escolhido.

8.1.1 ADR 1 — Adoção de comunicação assíncrona entre o sistema central e os sistemas externos

- **Contexto:**
 - No cenário selecionado, o nopCommerce necessita de interagir com sistemas externos como WMS e Shipping Service. Estas integrações podem estar sujeitas a atrasos, indisponibilidade temporária ou falhas, o que torna inadequada uma dependência forte de chamadas síncronas em fluxos críticos.
- **Decisão:**
 - Optou-se por introduzir comunicação assíncrona entre o nopCommerce e os sistemas externos, recorrendo a um Message Broker para troca de eventos relacionados com encomendas e respetivo processamento.
- **Alternativas consideradas:**
 - Integração síncrona direta entre o nopCommerce e os sistemas externos;
 - Processamento interno, no próprio nopCommerce, de toda a lógica de comunicação externa.
- **Justificação:**
 - A comunicação assíncrona permite reduzir o acoplamento entre componentes, melhorar a resiliência perante falhas temporárias e suportar consistência eventual entre contextos distintos. Além disso, contribui para que o sistema central permaneça operacional mesmo quando dependências externas se encontram degradadas.

8.1.2 ADR 2 — Introdução de um Order Integration Service dedicado

- **Contexto:**
 - A evolução para um contexto omnichannel exige a coordenação de fluxos entre o nopCommerce e múltiplos sistemas externos, com necessidade de gerir retries, correlação de mensagens, tratamento de falhas e lógica de integração específica.
- **Decisão:**
 - Optámos por introduzir um Order Integration Service como componente dedicado à integração entre o nopCommerce e os sistemas externos.
- **Alternativas consideradas:**
 - Manter toda a lógica de integração dentro do nopCommerce;
 - Integrar diretamente cada sistema externo com o nopCommerce sem um componente intermédio dedicado.

- **Justificação:**
 - A criação deste serviço permite separar responsabilidades, reduzir o acoplamento do sistema central e concentrar num único contexto as preocupações específicas da integração. Esta opção favorece a modularidade, a extensibilidade e a evolução futura da arquitetura, permitindo adicionar novos sistemas externos com menor impacto no núcleo da aplicação.
-

8.1.3 ADR 3 — Separação de ownership de dados entre contextos

- **Contexto:**
 - A arquitetura proposta envolve múltiplos contextos com responsabilidades distintas, nomeadamente experiência de comércio, gestão de encomendas, integração, stock e shipping. A partilha direta da base de dados entre estes contextos criaria dependências fortes e dificultaria a sua evolução independente.
 - **Decisão:**
 - Optou-se por definir ownership claro dos dados por contexto, evitando a partilha direta da base de dados entre o nopCommerce, o Order Integration Service e os sistemas externos.
 - **Alternativas consideradas:**
 - Utilização de uma base de dados partilhada entre múltiplos componentes;
 - Acesso direto aos dados de outros contextos para simplificar a implementação.
 - **Justificação:**
 - A separação de ownership de dados promove desacoplamento, coerência entre responsabilidades e maior independência evolutiva dos componentes. Esta decisão também está alinhada com as restrições do trabalho, que desencorajam a utilização de uma base de dados partilhada como atalho entre fronteiras extraídas.
-

8.2 Conclusão

As decisões apresentadas estabelecem a base estrutural da arquitetura alvo, garantindo coerência com o cenário selecionado, com os atributos de qualidade definidos e com os objetivos de evolução da plataforma. Estas decisões foram diretamente influenciadas pelos cenários de qualidade identificados, assegurando que a arquitetura proposta responde de forma consistente aos principais desafios do sistema. Em conjunto, estas ADRs permitem justificar as opções tomadas ao nível da integração, separação de responsabilidades e organização dos dados.

9. Plano de risco e estratégias de mitigação

A arquitetura proposta introduz novos componentes e mecanismos que, embora permitam responder aos desafios deste cenário, também levantam um conjunto de riscos arquiteturais. Assim, torna-se necessário identificar, previamente, esses riscos e definir estratégias de mitigação que permitam reduzir a sua incerteza.

9.1 Riscos arquiteturais

- **Risco 1 - Complexidade da comunicação assíncrona:**

A introdução de comunicação assíncrona e de um message broker aumenta a complexidade do sistema, nomeadamente ao nível da gestão de eventos, ordenação de mensagens e tratamento de falhas.

- **Risco 2 - Inconsistência temporária dos dados:**

A adoção de consistência eventual implica que diferentes sistemas possam apresentar estados divergentes durante períodos de tempo, o que pode afetar a experiência do utilizador.

- **Risco 3 - Fiabilidade das integrações externas:**

A dependência de sistemas externos (WMS, Shipping) pode introduzir falhas, atrasos ou comportamentos inesperados, impactando o processamento das encomendas.

- **Risco 4 - Observabilidade insuficiente em sistemas distribuídos:**

A falta de visibilidade sobre fluxos distribuídos pode dificultar a identificação de falhas e a análise do comportamento do sistema.

9.2 Estratégias de mitigação

Para mitigar os riscos identificados, propõem-se as seguintes estratégias de validação:

- **Mitigação da comunicação assíncrona:**
 - Implementação de um fluxo simples baseado em eventos (por exemplo: criação de encomenda → envio para integração);
 - Verificação do correto processamento de mensagens e comportamento em caso de falha.

 - **Mitigação da consistência eventual:**
 - Simulação de atrasos em sistemas externos;
 - Verificação da existência de estados intermédios corretos no nopCommerce;
 - Confirmação da atualização final consistente dos dados.

 - **Mitigação da fiabilidade das integrações:**
 - Testes com falhas simuladas em sistemas externos;
 - Verificação de mecanismos de retry e ausência de perda de mensagens;
 - Garantia de idempotência no processamento.

 - **Mitigação da observabilidade:**
 - Instrumentação do sistema com logs, métricas e tracing distribuído;
 - Monitorização de um fluxo completo (ex: processamento de encomenda);
 - Identificação de pontos de falha e análise de latência entre componentes.
-

9.3 Conclusão

Em jeito de conclusão do capítulo, a identificação destes riscos e a definição de estratégias de mitigação permitem reduzir a incerteza associada à arquitetura proposta, garantindo que as principais decisões podem ser testadas e ajustadas de forma iterativa antes da sua adoção completa.

10. Evolution Roadmap

A evolução da arquitetura proposta deve ser realizada de forma incremental, garantindo a continuidade do funcionamento do sistema e reduzindo o risco associado à introdução de novos componentes. Assim, define-se um conjunto de fases de implementação que permitem transitar progressivamente da arquitetura atual para a nova arquitetura correspondente aos nossos objetivos.

10.1 Fases de implementação

Posto isto, a evolução do sistema pode ser organizada nas seguintes fases:

- **Fase 1 — Introdução de observabilidade:**

Implementação de mecanismos de observabilidade no nopCommerce, incluindo logs, métricas e tracing distribuído, permitindo obter visibilidade sobre o comportamento do sistema e preparar a base para futuras alterações.

- **Fase 2 — Introdução do Message Broker:**

Integração de um message broker, permitindo suportar comunicação assíncrona e preparar o sistema para a troca de eventos.

- **Fase 3 — Criação do Order Integration Service:**

Desenvolvimento de um serviço dedicado à integração com sistemas externos, inicialmente com suporte a um fluxo simples (por exemplo: processamento de encomendas).

- **Fase 4 — Integração com sistemas externos:**

Ligação progressiva ao WMS e ao Shipping Service, começando por cenários simples e evoluindo para fluxos mais completos.

- **Fase 5 — Evolução para consistência eventual:**

Adaptação do modelo de estados no nopCommerce para suportar estados intermédios e sincronização assíncrona com sistemas externos.

10.2 Ordem de evolução

A ordem de implementação proposta segue uma abordagem incremental, iniciando-se com a introdução de capacidades transversais (observabilidade), seguida da infraestrutura de comunicação (message broker) e, posteriormente, da lógica de integração (Order Integration Service).

Esta sequência permite validar cada componente de forma isolada, reduzindo o risco e facilitando a identificação de problemas em fases iniciais.

10.3 Coexistências durante a transição

Um dos pontos mais importantes é, durante o processo de evolução, garantir a coexistência entre a arquitetura atual e os novos componentes introduzidos:

- O nopCommerce continuará a operar de forma autónoma enquanto os novos mecanismos são progressivamente integrados;
- As integrações com sistemas externos poderão coexistir em modo síncrono e assíncrono durante fases intermédias;
- O Order Integration Service poderá ser introduzido inicialmente apenas para determinados fluxos, mantendo os restantes no sistema central.

Esta abordagem permite uma transição gradual, minimizando o impacto no funcionamento do sistema e garantindo a continuidade da experiência do utilizador.

10.4 Conclusão

Concluindo, a definição deste roadmap permite estruturar a evolução da arquitetura de forma progressiva e controlada, assegurando que cada etapa contribui para a validação da solução e para a redução dos riscos associados à mudança.

11. Feasibility Spike

Tendo em conta a análise realizada às mudanças a efetuar do ponto de vista da arquitetura atual da plataforma nopCommerce e olhando também para o nosso cenário escolhido, decidimos estruturar e definir um feasibility spike com o objetivo de clarificarmos a parte mais arriscada de entre as alterações propostas, que corresponde à integração assíncrona e respetiva comunicação entre o sistema central e componentes externos.

11.1 Objetivo

Portanto, o objetivo deste spike é validar a viabilidade da utilização de comunicação assíncrona, baseada em eventos, para integração entre o nopCommerce e sistemas externos, garantindo, então, suporte a resiliência, desacoplamento e consistência eventual.

11.2 Descrição

O spike consiste na implementação de um fluxo simplificado de processamento de encomendas, incluindo:

- Emissão de um evento de criação de encomenda a partir do nopCommerce;
- Envio desse mesmo evento para um message broker;
- Consumo do evento por um componente externo simulado (por exemplo: Order Integration Service);
- Processamento do evento e simulação de resposta (por exemplo: confirmação ou falha);
- Registo de logs e métricas associadas ao fluxo.

Este cenário permite validar o comportamento do sistema em condições normais e em situações de falha, nomeadamente através da simulação da indisponibilidade do componente externo.

11.3 Resultados esperados

Com este feasibility spike, espera-se:

- Confirmar a correta emissão e consumo de eventos;
- Verificar o desacoplamento entre o sistema central e os componentes externos;
- Avaliar o comportamento do sistema perante falhas simuladas;
- Validar a utilização de mecanismos de retry e consistência eventual;
- Obter visibilidade sobre o fluxo através de logs, métricas e tracing.

11.4 Conclusão

Concluindo, este feasibility spike permite validar uma das decisões arquiteturais mais críticas da nossa proposta de solução, reduzindo o risco associado à introdução de comunicação assíncrona e garantindo maior confiança na implementação da arquitetura alvo.

12. Conclusão

Em síntese, o trabalho desenvolvido permitiu analisar a arquitetura atual da plataforma nopCommerce e definir uma proposta de evolução coerente com o cenário selecionado, o **Omnichannel Commerce Core**. A partir da análise do estado atual do sistema, da identificação de motivações de negócio e arquiteturais, da definição de cenários de qualidade e da aplicação do método ADD, foi possível estruturar uma arquitetura alvo orientada à integração com sistemas externos, à resiliência a falhas e à consistência eventual.

Ao longo desta primeira parte do projeto, foram igualmente identificadas decisões arquiteturais fundamentais, riscos associados à solução proposta e respectivas estratégias de mitigação, um roadmap de evolução e um feasibility spike para validação da componente mais crítica da arquitetura. Desta forma, este trabalho estabelece uma base sólida para a fase seguinte do projeto, permitindo avançar para a implementação e demonstração da solução de forma mais fundamentada, controlada e coerente com os objetivos definidos.